Realcryptik: Autonomous Election Verification with Zeeperio

Anonymized for review

Abstract. In this paper, we are interesting in whether an autonomous smart contract can verify an election. In this paper, we revisit the Eperio election backend that enables end-to-end verifiability (E2E-V) for a variety of ballot styles. Eperio proofs simple enough to verify with spreadsheet software. The roadblock to on-chain verification of Eperio proofs is the data size, which is linear in the number of voters. We propose Zeeperio, which replaces Eperio's cut-and-choose proof with a custom succinct zero-knowledge argument. The size of a Zeeperio proof is constant in the number of ballots and voters in an election, as is the time to verify it. We implement Zeeperio and demonstrate its proofs can be verified by a smart contract running on Ethereum.

Keywords: end-to-end verifiable voting, zero-knowledge proofs, zk-SNARKs, polynomial interactive oracle proofs, smart contracts

1 Introduction

Decades of research on end-to-end verifiable (E2E-V) elections have shown strong cryptographic guarantees can be realized for protecting the integrity of elections and the secrecy of voter ballots [15]. In more recent years, pilot projects and deployments of E2E-V platforms have also demonstrated that these systems are usable and scalable to an appropriate degree [2, 8, 4, 11]. Despite these efforts, the vendors of systems for governmental elections continue to run unverifiable elections where the content of individual ballots can be determined by at least one trusted entity in the platform.

In this paper, we contribute to a broader discussion the cryptographic voting community has been having for some number of years: do we compromise our ideals and offer watered-down systems that the industry might find more palatable if they offer tangible security advantages over the status quo? We relate this to the term Realpolitik, coined in 19th-century Germany to describe statecraft based on pragmatic calculations rather than ideology, and propose *Realcryptik* to describe such a pragmatic approach to election verification.

Flavours of Realcryptik already permeate the research literature. Despite opposition to internet voting, Helios [1] is perhaps the most used E2E-V voting technology. While researchers argue coercion resistance is essential for internet voting, Helios provides none. Scantegrity II [9] was the first E2E-V system to see governmental use. It is built around the compromise that in a paper ballot

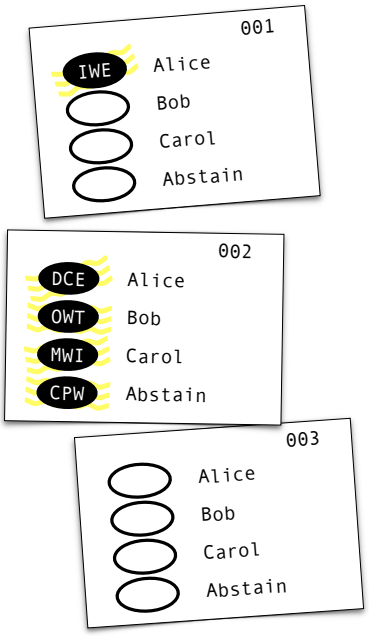
system, it is acceptable for optical scanner to learn how the voter voted, sidestepping the complex ballot mechanics its predecessors to avoid this. In recent years, serious discussions have arose around whether E2E-V should be largely abandoned in favor of risk limiting audits? Should we affirm the use of ballot marking devices when we know most voters do not carefully check their output?

We propose a different compromise in this paper: let EAs and their vendors see the raw ballots, while protecting them from all outsiders (receipt-freeness). For the EA, we still rely on legal protections implemented through procedures but not offer cryptographic protection.

The research question of this paper is how to modify the cryptography of an E2E-V voting system to provide integrity, without the need to maintain ballot secrecy from the EA (but still provide receipt-freeness otherwise). The simpler threat model means we can streamline the cryptography, in particular we can prove that encrypted (or committed in our case) ballots are tallied correctly rather than computing the tally under encryption. This shifts the protocol away from multiparty computation and threshold cryptography to essentially an all encompassing zero knowledge proof (ZKP). While ZKPs have been used in voting systems from almost their invention in 1980s and onward, we focus on modern succinct ZKPs called zk-SNARKs.

Zeeperio. We apply this modification to an existing E2E-V back-end called Eperio [12]. Eperio is a simplification of Scantegrity II's backend (known as a switchboard) and is design to be simple enough that a non-expert validator can audit the election using a spreadsheet, such as MS Excel. Eperio's security was studied in detail using formal methods at CCS 2021 paper [3]. By replacing Eperio's cut-and-choose based proof with a zk-SNARK, it obviously is no longer verifiable by hand. However this is a deliberate choice. Traditionally voters would either have to verify an election themselves or rely on a trusted entity (political party, watchdog) to perform this verification—hence the need to simplify elections to a spreadsheet. Today we have systems like Ethereum that provide a peer-to-peer network that is effectively validating that computational outputs of arbitrary code, called smart contracts, are correct. Ethereum protects assets with a market capitalization of $100+B USD and it can protect election results too, as long as the results are verifiable within its very restricted limits (known as gas consumption). If the verification task can be reduced to a succinct (zk-SNARK) proof, voters do not need check the proof at all.

Our contribution is designing a custom succinct argument for Eperio that we call Zeeprio. Our protocol uses polynomial interactive oracle proofs (PIOPs), which have been used to build zk-SNARKs for any arithmetic circuit (and thus any computation), the best known system being Plonk [13]. The size of a Zeeperio proof is constant in the number of ballots and voters in an election, as is the time to verify it. It is compact enough for autonomous verification by a smart contract on a blockchain, such as Ethereum. Zeeperio proofs are linear (as opposed to constant) in the number of candidates, but this seems acceptable as reading the final tally or checking an audited ballot is linear in the number of candidates.



Index	Ballot	Position	Code	Audit	Mark	Selection
0	001	1	IWE	0	1	Alice
1	001	2	XJS	0	0	Bob
2	001	3	NIP	0	0	Carol
3	001	4	QEM	0	0	Abstain
4	002	1	DCE	1	0	Alice
5	002	2	OWT	1	0	Bob
6	002	3	MWI	1	0	Carol
7	002	4	CPW	1	0	Abstain
8	002	1	WNJ	0	0	Alice
9	003	2	IES	0	0	Bob
10	003	3	CNE	0	0	Carol
11	003	4	PCS	0	0	Abstain
12	Pad		0	0	0	
13	Pad		0	0	0	
14	Pad		0	0	0	
15	Pad		0	0	0	

Fig. 1. An Eperio/Zeeperio table illustrated with Scantegrity II ballots which reveal a hidden confirmation code for each selection marked. Eperio commits to columns: ballot, position, mark, selection (it technically does not work with Scantegrity II so selection will encode a Scantegrity I or Pret a Voter layout). Many copies are made, shuffled row-wise, and based on random coin tosses, the EA opens {ballot, position, mark} or they open {mark, selection} enabling receipt checks and tally checks respectively. We omit how Eperio handles print audits. In Zeeperio, the table is not shuffled so the white columns are public information and not committed to. The pink columns are padded (purpose described below) and committed to. The EA uses these commitment to demonstrate certain properties (constraints) hold for each pink column.

2 Background and Related Work

2.1 Eperio

We assume the reader is already familiar with end-to-end verifiable (E2E-V) voting protocols which allow for: (i) an election authority to prove an election tally is computed correctly without learning individual voter choices, and for (ii) voters to check their individual ballots are included unmodified in the tally without being able to reveal how they voted to anyone else. Eperio is an E2E-V tallying protocol (or 'backend') that composes with several styles of paper ballots, including Pret a Voter and Scantegrity I. A brief description of Eperio is provided in Figure 1 with a Pret a Voter style ballot.

In addition to succinctness, we also clean up a few issues that were not fully resolved in Scantegrity II and Eperio:

- As pointed about by Basin *et al.* [3], Eperio does not prove ballots were not overvoted; Zeeperio does (§3.7).
- As acknowledged by Essex *et al.* [12], Eperio does not support Scantegrity II ballots; Zeeperio does (§4.2).
- As acknowledged by Chaum *et al.* [9], Scantegrity II supports dispute resolution but it requires all confirmation codes on the ballot to be revealed. Zeeprio offers a simple proof of incorrect code without revealing information about all codes (§4.3).

A remaining issue, pointed out by Basin et al [3], are clash attacks [17] which are not explicitly addressed by Eperio. However they are applicable to all major E2E-V voting systems and are addressed through strict polling place policies that denote which ballot is given to which voter.

2.2 Succinct Proofs

While zero knowledge proofs/arguments (ZKPs) date back to the 1980s [14, 6], classic ZKPs produce proofs that are at least linear in the size of the statement being proven. By contrast, succinct zero knowledge arguments (SNARKs) are sublinear, typically with a logarithmic or constant proof size. A Polynomial Interactive Oracle Proof (PIOPs) is a pattern that several SNARKs have in common [18, 10, 7]. In this pattern, the prover specifies polynomials and verifiers challenge the prover to open these polynomials at random points. PIOP does not say *how* to make a SNARK; it is merely a generic pattern.

To realize a PIOP, the prover must send a succinct representation of each polynomial from the prover to the verifier, which commits the prover to those polynomials. When opening a polynomial, the verifier must be certain it is the opening of the same polynomial that was committed to. PIOPs rely on a cryptographic primitive called a polynomial commitment scheme (PCS) to handle this. The rest of a PIOP protocol does not use any cryptography. A number of PCSs have been proposed, the most prominent are KZG (very short, based on bilinear pairings, requires a trusted setup) and (DEEP-)FRI (longer, based on hash functions, conjectured to be post-quantum secure, no trusted setup). In Zeeperio, we use KZG [16] because it integrates well with Ethereum.

KZG uses a universal trusted setup: a secret value τ is chosen at random, used to compute a structured reference string (SRS) $\left\langle g^{\tau}, g^{\tau^2}, g^{\tau^3}, \ldots \right\rangle$ in a pairing-friendly group defined by generator g, and then erased. Knowledge of τ after the setup is sufficient to fake KZG openings and thus forge false ZKP transcripts. However the setup can be run as an open participation multiparty computation where participants prove their contribution is correct and only one honest participant must forget τ for the SRS to be secure. Ethereum ran an open trusted setup in 2023, accepting 100K contributions over 208 days. A single SRS, such

as this, can be shared by everyone wanting to do a KZG commitment (known as a universal trusted setup).

In practice, zk-SNARKs for a specific computation can be deployed in different ways: (a) written in a high-level language and compiled to VM instructions, (b) written in special markup languages and compiled into arithmetic circuits, or (c) written as a custom arithmetic circuit. In each case, these representations are fed into a general-purpose proof system for circuits or zkVMs. Each level of abstraction adds overhead and costs to the prover. Our approach is 'closer to the metal.' We do not use circuits or Plonk itself. We instead go underneath what Plonk provides and use the same kind of building blocks it uses to make a custom PIOP argument for Zeeperio.

2.3 Smart Contracts

Smart contracts are programs deployed on a blockchain, such as Ethereum. They only execute when a user calls a function. The blockchain's consensus enforces that the function call executes correctly, and the state of the code is updated accordingly. In Zeeprio, a smart contract will accept an election proof and run the verification of it, reporting if it is correct or incorrect. Since smart contracts can implement arbitrary code, they can support cryptographic operations however this approach is usually prohibitively expensive. Ethereum offers cheaper 'pre-compiles' for pairing-based operations on the elliptic curve `alt-bn128`. On Ethereum's testnet, support for curve `bls12-381` has been staged for future deployment, as well as KZG-specific operations over `bls12-381`, using the aforementioned SRS. Zeeperio on `alt-bn128` or `bls12-381`

3 Zeeperio

Parameters. Let $\mathcal{B}$ be the number of printed ballots and $\mathcal{C}$ the number of candidates per ballot. Let $\mathcal{P} = \mathcal{BC}$ be the number of ballot positions. Let n be the padded column length (a power of two) with $n \geq \mathcal{P}$ (the need is explained below). Let $\mathbb{F}_q$ be a finite field of size q. A ballot has ID $b \in \{0, \ldots, \mathcal{B}-1\}$ and occupies a *block*: $I_b = \{\, b\mathcal{C},\, b\mathcal{C}+1,\, \ldots,\, b\mathcal{C}+(\mathcal{C}-1)\,\} \subseteq \{0, \ldots, n-1\}$.

Walkthrough of a constraint. Notice from Figure 1, the mark column can only contain a 0 or 1. This is one small fact about what constitutes a valid election table. With ZKPs, we call this a *constraint*. Zeeperio consists of 16 similar constraints that all must hold in any valid election table. Here, we will demonstrate a complete argument for this single constraint (which we denote as c2.2 later) to demonstrate how a PIOP works. This constraint is adapted from [5].

The mark column can be considered a one-dimensional vector or array $A_{\mathsf{mark}}[i]$ of length $\mathcal{P}$. Elements in the array will be encoded as elements of $\mathbb{F}_q$, the exponent field of a pairing-friendly elliptic curve such as `alt-bn128` or `bls12-381`. Cryptographically speaking, the array $A_{\mathsf{mark}}[i]$ could thus hold any integer in

$[0, q-1]$ but the constraint says it is only a valid election table if it is further restricted to $[0, 1]$.

To prove the constraint, the prover (election authority) will send to the verifier (smart contract) a succinct (one group element) commitment (hiding and binding) to the entire mark column $A_{\mathsf{mark}}[i]$. While vector commitments exist, in a PIOP, the prover commits to $A_{\mathsf{mark}}[i]$ indirectly by first encoding the data from $A_{\mathsf{mark}}[i]$ into a univariate polynomial over $\mathbb{F}_q$ and then commits to this polynomial representation of $A_{\mathsf{mark}}[i]$ using a polynomial commitment scheme such as KZG.

There are a variety of ways to encode vectors into polynomials, and we will focus on the method we will use. The prover chooses a set of x-coordinates, such as $\{0, 1, 2, 3, \ldots\}$ which is called an evaluation domain $\mathcal{H}$. It then takes the data from $A_{\mathsf{mark}}[i]$ and forms a set of points $\{(0, A_{\mathsf{mark}}[0]), (1, A_{\mathsf{mark}}[1]), \ldots\}$. The prover interpolates a univariate polynomial through these points: $\mathsf{Mark}(X)$. It commits to $\mathsf{Mark}(X)$ using the polynomial commitment scheme, and sends the commitment to the verifier.

Interpolation is computationally expensive for provers, so they use an optimization. The evaluation domain is arbitrary and can be anything as long as the prover and verifier agree. Let $\omega \in \mathbb{F}_q$ generate a multiplicative subgroup of size n. Then set $H = \{\omega^0, \omega^1, \ldots, \omega^{n-1}\} \subseteq \mathbb{F}_q$. Doing so offers three advantages and one drawback. Interpolating a polynomial over this domain can be accomplished in $\mathcal{O}(n \log n)$ instead of $\mathcal{O}(n^2)$. Traversing the domain easy: the next index is the current index multiplied by ω and because it is closed under multiplication, the 'next' element after the last element ω^{n-1} wraps back to the first element $\omega^{n-1} \cdot \omega = \omega^n = \omega^0$. Finally, a polynomial that is 0 at each $X \in H$ has a simple closed form: $\mathsf{Z}_H(X) = X^n - 1$. The drawback is that the prover cannot use arrays of arbitrary length $\mathcal{P}$ and expect that an order n subgroup will just exist. The subgroups are based on the parameters of the elliptic curve and curves like `alt-bn128` and `bls12-381` are designed to offer subgroups of sizes $\{2, 2^2, 2^3, 2^4, \ldots\}$ up to 2^{12} for `alt-bn128` and 2^{32} for `bls12-381`.

In Zeeprio, if we have a column of length $\mathcal{P}$, we will zero pad it the nearest power of 2. Thus n is the padded column length (a power of two) with $n \geq \mathcal{P}$. For this example constraint (the mark column is binary), the presence of zero padding does not affect the constraint. However, for other constraints, we will have to explicitly handle the padding separately from the election table data.

At this point, the verifier has a polynomial commitment to $\mathsf{Mark}(X)$ which we will call K_{Mark}. How does the prover demonstrate it is binary? The prover will construct another polynomial as follows: $V(X) := \mathsf{Mark}(X)\big(\mathsf{Mark}(X) - 1\big)$. It will commit to $V(X)$ and send K_V to the verifier. If $\mathsf{Mark}(X)$ is binary for all $X \in H$, then $V(X)$ will be zero for all $X \in H$. If $\mathsf{Mark}(X)$ is not binary for at least one $X \in H$, $V(X)$ will be non-zero at the same X. We have converted the constraint $\mathsf{Mark}(X)$ is binary into:

1. $V(X)$ is correctly constructed from $\mathsf{Mark}(X)$
2. $V(X)$ it is zero for all $X \in H$. A polynomial that is is zero for all $X \in H$ is called a *vanishing polynomial* (on H).

To prove that $V(X)$ is correctly constructed from $\mathsf{Mark}(X)$, the verifier has K_{Mark} and K_V. The verifier will (or the prover using the strong Fiat-Shamir heuristic) choose a random evaluation point $\zeta \notin H$. Using the polynomial commitment scheme, the prover will open the polynomials at the value ζ sending $V(\zeta)$, $\mathsf{Mark}(\zeta)$, and opening proofs for each to the verifier. The verifier will check the opening proofs and check that $V(\zeta) := \mathsf{Mark}(\zeta)\big(\mathsf{Mark}(\zeta) - 1\big)$. Just because the relation holds at the point ζ, what does that say about it holding at each $X \in H$? The answer is that it overwhelmingly holds due to the Schwartz-Zippel lemma, which we will not review further here.

Next, the prover will demonstrate that $V(X)$ vanishes on H. Recall that $\mathsf{Z}_H(X) = X^n - 1$ is a vanishing polynomial on H. The prover will construct another polynomial $Q(X) := V(X)/\mathsf{Z}_H(X)$, commits to $Q(X)$, and sends the commitment K_Q to the verifier. If and only if $V(X)$ is zero at every point of H, $\mathsf{Z}_H(X)$ exactly divides $V(X)$ and results in a proper polynomial $Q(X)$, otherwise $Q(X)$ will be a rational function not reducible to a polynomial. $Q(X) := V(X)/\mathsf{Z}_H(X)$ can be rearranged as $V(X) - Q(X) \cdot (X^n - 1) = 0$. Using ζ from above, the prover sends $Q(\zeta)$ and its opening proof. The verifier checks $V(\zeta) - Q(\zeta) \cdot (\zeta^n - 1) = 0$. By Schwartz–Zippel, if this equality holds at a uniformly random $\zeta \notin H$, then except with negligible probability it holds as a polynomial identity, which implies that $V(X)$ vanishes on H. □

3.1 Zeeperio Constraints

Zeeperio is made up of 16 constraints. Each constraint follows a similar pattern to the example constraint: a set of polynomials is committed to such that a relationship between them will be a vanishing polynomial if and only if the constraint is true. Once the constraint is converted into a vanishing polynomial, proving the vanishing polynomial is constructed correctly and that it actually vanishes is identical to the process used above.

In fact, given a set of vanishing polynomials $\{V_1(X), V_2(X), \ldots, V_m(X)\}$ and their corresponding quotient polynomials, the check can be batched into a single check. The prover will form a 'super-polynomial' $\mathsf{Batch}(X)$ where each coefficient is one of the vanishing polynomials:

$$\mathsf{Batch}(Y) = \sum_{i=1}^{m} Y^i \Big(V_i(X) - Q_i(X)\, \mathsf{Z}_H(X) \Big)$$

A polynomial where every coefficient is 0 is a zero-polynomial. Therefore the verifier (or prover via strong Fiat-Shamir) will choose a random $\zeta \in \mathbb{F}_q \notin H$ as before and set $X = \zeta$, which overwhelmingly makes every coefficient zero. To see if every coefficient is zero, it will also choose a random $\alpha \in \mathbb{F}_q$ and set $Y = \alpha$. If every $V_i(X)$ vanishes, then $\mathsf{Batch}(Y)$ is a zero polynomial and:

$$\mathsf{Batch}(\alpha) = \sum_{i=1}^{m} \alpha^i \Big(V_i(\zeta) - Q_i(\zeta)\, \mathsf{Z}_H(\zeta) \Big) = 0.$$

Table 1. A set of polynomials that can be computed before the beginning of the election. $\mathsf{Sel}_b(X)$ is the one exception, it will not be precomputed but rather computed during a receipt check just for the one ballot b that the voter wants to check.

Description	**Polynomial**
Polynomial that is zero on H	$\mathsf{Z}_H(X) = X^n - 1$
Mask that is 0 on all of H except possibly at $X = \omega^0$	$\mathsf{Mask}_0(X) = \dfrac{\mathsf{Z}_H(X)}{X - \omega^0}$
Mask that is 0 on all of H except possibly at $X = \omega^{n-1}$	$\mathsf{Mask}_{n-1}(X) = \dfrac{\mathsf{Z}_H(X)}{X - \omega^{n-1}}$
Polynomial that is 0 up to padding $[0, \mathcal{P}-1]$	$\mathsf{Z}_{\mathsf{head}}(X) = \prod_{i=0}^{\mathcal{P}-1} \left(X - \omega^i\right)$
Polynomial that is 0 for the last $\mathcal{C}$ rows (zeros at $\omega^{n-1}, \ldots, \omega^{n-\mathcal{C}}$)	$\mathsf{Mask}_{\mathsf{tail}}(X) = \prod_{t=0}^{\mathcal{C}-1} \left(X - \omega^{n-1-t}\right)$
Lagrange basis over H used below	$L_i(X) = \prod_{\substack{0 \le j < n \\ j \ne i}} \dfrac{X - \omega^j}{\omega^i - \omega^j}$
Polynomial with a 1 at the start of every ballot (indices $i \equiv 0 \pmod{\mathcal{C}}$ up to padding) and 0 elsewhere	$\mathsf{Sel}_{\mathsf{blk}}(X) = \sum_{j=0}^{\lfloor n/\mathcal{C} \rfloor - 1} L_{j\mathcal{C}}(X)$
Polynomial that is 1 for all rows belonging to ballot-b ($I_b = \{b\mathcal{C}, \ldots, b\mathcal{C} + \mathcal{C} - 1\}$) and 0 elsewhere	$\mathsf{Sel}_b(X) = \sum_{k=0}^{\mathcal{C}-1} L_{b\mathcal{C}+k}(X)$

3.2 Public Polynomials

Zeeperio uses several public helper polynomials that depend only on the election 'shape' (e.g., n, $\mathcal{B}$, $\mathcal{C}$, and hence $\mathcal{P} = \mathcal{BC}$), and not on any voter choices or on the contents of the committed columns. They are enumerated in Table 1. Some of these helpers are linear or superlinear in n to construct (for example, explicit products or explicit sums of Lagrange basis polynomials). This does not undermine the succinctness claims of Zeeprio: the pre-computation is offline, public, and independent of the witness. In systems like Plonk, these types of polynomials are considered as a 'verifier's key.' In practice, election administrators can standardize on a small set of template sizes, and these polynomials can be precomputed once for each election template. If an election uses fewer than the template's maximum number of ballots or candidates, the remaining rows are constrained to be empty: extra ballots are unused and extra candidates receive zero votes, which is directly reflected in the published tally.

3.3 Election Specific Polynomials

The Eperio tables for an election consist of three (padded) columns, which will be encoded into polynomials evaluated over H. We call the polynomials $\mathsf{Code}(X)$,

$\mathsf{Audit}(X)$, and $\mathsf{Mark}(X)$. How each index maps to a unique ballot ID and candidate is provided as a public template, and columns will be sorted by ballot ID and then candidate (see Figure 1). The final tally is a vector $\mathsf{Tally}[0..\mathcal{C}-1] \in \mathbb{F}_q^{\mathcal{C}}$.

Across all the constraints, we will use an additional 5 'helper' polynomials. We will introduce them as needed, however they consist of two (additive, suffix) accumulator polynomials: $\mathsf{Acc}_{\mathsf{Audit}}(X)$ and $\mathsf{Acc}_{\mathsf{Mark}}(X)$, where $\mathsf{Acc}_{\mathsf{Audit}}(\omega^i) = \sum_{t=i}^{n-1} \mathsf{Audit}(\omega^t)$; likewise for $\mathsf{Mark}(X)$. We also define a 'stridded' accumulator $\mathsf{Acc}_{\mathsf{Vote}}(X)$ that we will describe with constraints 4. Finally we also define two block sum helper polynomials: $\mathsf{BlkSum}_{\mathsf{Audit}}(X) = \sum_{k=0}^{\mathcal{C}-1} \mathsf{Audit}(\omega^k X)$ and $\mathsf{BlkSum}_{\mathsf{Mark}}(X) = \sum_{k=0}^{\mathcal{C}-1} \mathsf{Mark}(\omega^k X)$.

3.4 Audit Constraints

Due to limited space in the paper, we cannot fully explain each of the 16 constraints. Instead we provide some intuition for them.

$$
\begin{aligned}
V_{c1.1}(X) &:= \mathsf{Audit}(X)\,\mathsf{Z}_{\mathsf{head}}(X) \\
V_{c1.2}(X) &:= \mathsf{Audit}(X)\big(\mathsf{Audit}(X) - 1\big) \\
V_{c1.3}(X) &:= \big(\mathsf{Sel}_{\mathsf{blk}}(X)\mathsf{BlkSum}_{\mathsf{Audit}}(X)\big)\big(\mathsf{Sel}_{\mathsf{blk}}(X)\mathsf{BlkSum}_{\mathsf{Audit}}(X) - \mathcal{C}\big) \\
V_{c1.4}(X) &:= \big(\mathsf{Acc}_{\mathsf{Audit}}(X) - \mathsf{Audit}(X) + \mathsf{Acc}_{\mathsf{Audit}}(\omega X)\big) \cdot (X - \omega^{n-1}) \\
V_{c1.5}(X) &:= \big(\mathsf{Acc}_{\mathsf{Audit}}(X) - \mathsf{Audit}(X)\big) \cdot \mathsf{Mask}_{n-1}(X) \\
V_{c1.6}(X) &:= \big(\mathsf{Acc}_{\mathsf{Audit}}(X) - \mathsf{Sum}_{\mathsf{Audit}\times\mathcal{C}}\big) \cdot \mathsf{Mask}_0(X)
\end{aligned}
$$

The audit column is well-defined for an election if all padding bits have the value 0 (c1.1) and the column is otherwise binary (c1.2). Each ballot spans $\mathcal{C}$ positions, so the audit column should contain a contiguous block of $\mathcal{C}$ ones for an audited ballot, and a block of $\mathcal{C}$ zeros for unaudited ballots. The mechanism is to compute a block-sum over each ballot, $\mathsf{BlkSum}_{\mathsf{Audit}}(X) = \sum_{k=0}^{\mathcal{C}-1} \mathsf{Audit}(\omega^k X)$, then restrict attention to ballot-start indices using $\mathsf{Sel}_{\mathsf{blk}}(X)$. For a well-formed audit column, the filtered block-sum is always either 0 (not audited) or $\mathcal{C}$ (audited), and never an intermediate value. Note that after choosing ζ, in order to check $\mathsf{BlkSum}_{\mathsf{Audit}}(X)$ is well-formed, the verifier requires $\mathcal{C}$ opened points from $\mathsf{Audit}(\omega^0\zeta)$ to $\mathsf{Audit}(\omega^{\mathcal{C}-1}\zeta)$, so this check is linear in $\mathcal{C}$ (but more importantly constant in $\mathcal{P}$ and n).

The EA will assert the number of print audits in the election as the integer $\mathsf{Sum}_{\mathsf{A}}$. Since each print audit ballot contains a block of $\mathcal{C}$ 1's, the number of 1's in $A(X)$ must be $\mathsf{Sum}_{\mathsf{A}\times\mathcal{C}} = \mathsf{Sum}_{\mathsf{A}} \cdot \mathcal{C}$. To reason about the sum across a polynomial on H, many protocols use accumulator polynomials [13, 5]. $\mathsf{Acc}_{\mathsf{Audit}}(X)$ is intended to hold a running suffix-sum of the audit column down the domain order (including padding, which is forced to be 0 by c1.1):

$$
\begin{aligned}
\mathsf{Acc}_{\mathsf{Audit}}(\omega^i) &:= \mathsf{Audit}(\omega^i) + \mathsf{Acc}_{\mathsf{Audit}}(\omega^{i+1}) \quad \text{for } i = 0, 1, \ldots, n-2, \\
\mathsf{Acc}_{\mathsf{Audit}}(\omega^{n-1}) &:= \mathsf{Audit}(\omega^{n-1}).
\end{aligned}
$$

Constraint c1.4 and c1.5 check each line of this recurrence respectively, guaranteeing that $\mathsf{Acc}_{\mathsf{Audit}}(X)$ is formed correctly. The first value in $\mathsf{Acc}_{\mathsf{Audit}}(X)$ holds the sum, so c1.6 ensures this value is equal to $\mathsf{Sum}_{\mathsf{A}\times\mathcal{C}}$.

3.5 Mark Constraints

$$
\begin{aligned}
V_{c2.1}(X) &:= \mathsf{Mark}(X)\,\mathsf{Z}_{\mathsf{head}}(X) \\
V_{c2.2}(X) &:= \mathsf{Mark}(X)\big(\mathsf{Mark}(X)-1\big) \\
V_{c2.3}(X) &:= \big(\mathsf{Sel}_{\mathsf{blk}}(X)\mathsf{BlkSum}_{\mathsf{Mark}}(X)\big)\big(\mathsf{Sel}_{\mathsf{blk}}(X)\mathsf{BlkSum}_{\mathsf{Mark}}(X)-1\big) \\
V_{c2.4}(X) &:= \big(\mathsf{Acc}_{\mathsf{Mark}}(X)-\mathsf{Mark}(X)+\mathsf{Acc}_{\mathsf{Mark}}(\omega X)\big)\cdot(X-\omega^{n-1}) \\
V_{c2.5}(X) &:= \big(\mathsf{Acc}_{\mathsf{Mark}}(X)-\mathsf{Mark}(X)\big)\cdot\mathsf{Mask}_{n-1}(X) \\
V_{c2.6}(X) &:= \big(\mathsf{Acc}_{\mathsf{Mark}}(X)-\mathsf{Sum}_{\mathsf{Mark}}\big)\cdot\mathsf{Mask}_{0}(X)
\end{aligned}
$$

The constraints on the mark column parallel the constraints on the audit column. Padding is zero (c2.1), the column is binary (c2.2), and the total number of votes matches the sum announced by the election authority $\mathsf{Sum}_{\mathsf{Mark}}$ (c2.4–c2.5). The main difference is constraint c2.3. For each ballot in the audit column (previous section), the ballot's block is either all 0 or all 1, summing to either 0 or $\mathcal{C}$. For the vote column (this section), each ballot block is either all 0 (not voted) or contains a single 1 (voted). Since it is binary (c2.2), the EA can demonstrate that the sum of each ballot block is either 0 or 1, which is c2.3. We assume that over-voted ballots are rejected by the scanner or automatically converted into print audit ballots.

3.6 Cross-Column Constraints

$$V_{c3.1}(X) := \mathsf{Audit}(X)\,\mathsf{Mark}(X)$$

A final constraint demonstrates that no ballot is both voted and audited. This is simple: an unvoted ballot will have all 0's in its ballot block, while an unaudited ballot will have the same. Therefore, the Hadamard product (element-wise multiplication) of the audit column with the mark column will be 0 in every element if it is properly formed. In the malicious case, if a ballot is both voted and audited, at least one element of the ballot block will contain a 1 in both columns, and the product of the columns will not be 0 at that element, thus $V_{c3.1}(X)$ will not vanish.

3.7 Tally Constraints

$$
\begin{aligned}
V_{c4.1}(X) &:= \big(\mathsf{Acc}_{\mathsf{Vote}}(X) - \mathsf{Mark}(X) + \mathsf{Acc}_{\mathsf{Vote}}(\omega^{\mathcal{C}} X)\big) \cdot \mathsf{Mask}_{\mathsf{tail}}(X) \\
V_{c4.2}(X) &:= \big(\mathsf{Acc}_{\mathsf{Vote}}(X) - \mathsf{Mark}(X)\big) \cdot \frac{\mathsf{Z}_H(X)}{\mathsf{Mask}_{\mathsf{tail}}(X)} \\
V_{c4.3,j}(X) &:= \big(\mathsf{Acc}_{\mathsf{Vote}}(X) - \mathsf{Tally}[j]\big) \cdot \frac{\mathsf{Z}_H(X)}{X - \omega^j}
\end{aligned}
$$

The EA will announce the tally as a list of vote totals for each of the $\mathcal{C}$ candidates: $\mathsf{Tally}[j]$ for $j \in \{0, \ldots, \mathcal{C}-1\}$. The marks for candidate i will be in every i-th element of $\mathsf{Mark}(X)$ over H. The EA will build an additive, suffix accumulator $\mathsf{Acc}_{\mathsf{Vote}}(X)$ however it will be *strided*:

$$
\mathsf{Acc}_{\mathsf{Vote}}(\omega^i) := \sum_{t=0}^{\lfloor \frac{n-1-i}{\mathcal{C}} \rfloor} \mathsf{Mark}\big(\omega^{i+t\mathcal{C}}\big), \qquad i \in \{0, \ldots, n-1\}.
$$

The result is that $\mathsf{AccVote}(\omega^c)$ is equal the total number of marks appearing at indices congruent to $c \bmod \mathcal{C}$ across all ballots (and padding). Note that there is no assumption that $c|n$; the sum for some candidates may be one element longer than others. Constraint c4.1 demonstrates the accumulator is built correctly, assuming it starts with the right values. Constraint c4.2 demonstrates that the starting values are correct. Constraint c4.3 demonstrates that the published tally matches the sum of each candidate's vote. Constraint c4.3 is run $\mathcal{C}$ times, once for each candidate's total votes, which will be contained in the first $\mathcal{C}$ elements of $\mathsf{AccVote}(X)$ on H.

4 Voter Checks

Eperio, like many voting systems, splits the E2E-V proof into two parts: a universal verifiable proof that the tally is correct, and a voter-verifiable proof that receipts are correct. All constraints from Section 3 correspond to the former proof, whereas in this section we describe the voter-verifiable checks.

4.1 Print Audit

The voter provides ballot ID b, which determines the public index set $I_b = b\mathcal{C}, \ldots, b\mathcal{C} + \mathcal{C} - 1$. The EA opens the committed polynomials $\mathsf{Audit}(X)$, $\mathsf{Mark}(X)$, and $\mathsf{Code}(X)$ at the $\mathcal{C}$ points $\omega^i{}_{i \in I_b}$ and returns those evaluations together with opening proofs (for KZG, these can be batched in a single multipoint opening proof).

4.2 Receipt Check

At a first glance, it might seem the EA can simply open $\mathsf{Mark}(X)$ and $\mathsf{Code}(X)$ for the voter's ballot ID b. However, since these polynomials are ordered by

candidate, revealing the index of the mark reveals who the voter voted for. We analyzed three PIOP-friendly approaches to this, but for space considerations, we present only the best: using a permutation (briefly, the excluded alternatives are lookup arguments and a secret selector vector with summation).

First, the EA defines a public *ballot-ID column* $\mathsf{BallotID}(X)$ that encodes the ballot identifier on every ballot position: for each ballot $b \in \{0, \ldots, \mathcal{B}-1\}$ and each within-ballot offset $k \in \{0, \ldots, \mathcal{C}-1\}$, we set $\mathsf{BallotID}(\omega^{b\mathcal{C}+k}) = b$, and we set $\mathsf{BallotID}(\omega^i) = 0$ on all padding indices $i \geq \mathcal{P}$. Next, the EA samples a secret permutation π over $\{0, \ldots, n-1\}$ and uses it to define permuted columns by $\mathsf{F}^\pi(\omega^i) = \mathsf{F}(\omega^{\pi(i)})$. The EA then commits once (pre-election or once-per-election) to the shuffled columns $\mathsf{BallotID}^\pi(X)$, $\mathsf{Code}^\pi(X)$, and $\mathsf{Mark}^\pi(X)$, and proves once that each shuffled column is a permutation of its corresponding source column under the *same* secret permutation π. The permutation argument comes straight from the literature [13].[1]

For a per-receipt query, the voter provides ballot ID b. The EA finds, privately, an index t such that $\mathsf{BallotID}^\pi(\omega^t) = b$ and $\mathsf{Mark}^\pi(\omega^t) = 1$, and it returns the receipt code $c := \mathsf{Code}^\pi(\omega^t)$ together with a batched opening proof at the single point ω^t for the three committed polynomials $\mathsf{BallotID}^\pi(X)$, $\mathsf{Mark}^\pi(X)$, and $\mathsf{Code}^\pi(X)$. Revealing the index t is safe because π is secret, so t does not reveal the candidate-order position of the mark in the original (unshuffled) ballot block.

4.3 Dispute Resolution

If a receipt check returns a wrong confirmation code, it is possible that the EA switched the voter's vote to another candidate. However, the voter has proof of this: they know a different confirmation code that appears on the ballot. By contrast, a voter who wants to discredit an election result might claim spuriously that their confirmation code is incorrect (even though it is); however, they will not know a different valid confirmation on their ballot (beyond guessing) [9].

In a dispute, the voter provides a ballot ID b and a disputed confirmation code $c \in \mathbb{F}_q$. The EA's goal is to prove *non-membership*: that c does not appear anywhere among the $\mathcal{C}$ confirmation codes printed on ballot b, without revealing any of those codes. Let $I_b = \{b\mathcal{C}, \ldots, b\mathcal{C} + (\mathcal{C}-1)\}$ denote the (public) index set of ballot b, and let $\mathsf{Mask}^{(b)}_{\mathsf{ballot}}(X)$ be the public 0/1 mask that is 1 on I_b and 0 elsewhere on H. If c is a non-member, then $(\mathsf{Code}(\omega^i) - c)$ will be non-zero on each element in I_b. If and only if it is non-zero, an inverse element will exist such that $(\mathsf{Code}(\omega^i) - c) \cdot \mathsf{Inv}_b(\omega^i) = 1$. The EA introduces an auxiliary polynomial of asserted inverses $\mathsf{Inv}_b(X)$ on I_b and the constraint reduces the vanishing polynomial:

$$V_{\mathsf{dispute}}(X) \;:=\; \big((\mathsf{Code}(X) - c) \cdot \mathsf{Inv}_b(X) - 1\big) \cdot \mathsf{Mask}^{(b)}_{\mathsf{ballot}}(X)$$

[1] The only minor difference is that we are shuffling the random linear combination of three polynomials instead of two polynomials under the same permutation.

If c does appear on the ballot, say at position $\kappa \in I_b$, the EA will not be able to put a value in $\mathsf{Inv}_b(\kappa)$ that makes $V_{\mathsf{dispute}}(X)$ vanish.

5 Implementation and Performance Evaluation

Zeeperio is organized around two main pieces: an off-chain prover written in Rust and an on-chain verifier in Solidity. The Rust side is split into small libraries: one defines the core proof logic using the arkworks cryptography toolkit (for pairing-friendly curves and polynomial math), another wraps KZG commitment helpers, a shared crate holds the data types exchanged across the system, and a command-line tool ties them together for end users. The Solidity verifier is a Foundry project that encodes the `alt-bn128` elliptic-curve constants and routines to check different proof variants once they are submitted to a smart contract.

To satisfy the zero-knowledge property, the prover uses *hiding (or Pedersen)* KZG commitments, where it adds a blinding term derived from a $\gamma \cdot g$ SRS basis [16]. Additionally, the prover uses a technique—employed and popularized in Sonic [18]—we call the MBKM heuristic[2] which randomizes polynomials with additional off-domain points so that any bounded number of evaluations of the polynomial at random points leak zero information (in an information theoretical sense) about any other point on the polynomial.

Challenges are derived non-interactively with the strong Fiat-Shamir heuristic, hashing a serialized transacript of the proof's parameters, public inputs and commitments in a fixed order. The transcript yields (α, ζ) for the main proof and additional challenges for the inclusion/shuffle proof. Both prover and verifier must share *exact* serialization rules (little-endian field encoding and compressed group encoding), otherwise valid proofs will be rejected.

The bilinear pairing used to check a KZG opening proof is the most expensive on-chain operation. All openings that occur at the same evaluation point can be batched. As a result, the verifier performs roughly one pairing-based check (via an Ethereum precompile operation) per evaluation point, while the remaining work is less expensive glue logic.

For usability and integration, the verifier exposes three entrypoints (`verifyMain`, `verifyInclusion`, and `verifyReceipt`) corresponding to the full-election proof and smaller auxiliary checks, each consuming a single ABI-encoded proof object. The ABI payload is a structured record containing the public inputs, the list of polynomial commitments, and a list of opening batches (each batch specifies its evaluation point, the folding scalar γ, the set of commitments opened there, the claimed values, and the single aggregated opening proof). To keep decoding and bookkeeping lightweight, commitments and opened values are referenced by compact 16-bit labels rather than long identifiers. This lets the contract locate the required terms via simple scans over the decoded arrays, keeping the EVM-side logic small and auditable even for readers who are not Ethereum specialists.

[2] Variously referred to in the literature as blinders, vanishing-polynomial blinding, the Z_H-mask trick, and off-domain masking.

Table 2. Implementation metrics for $B = 100{,}000$, $C = 8$

Metric	Main	Inclusion	Receipt
Prover time (s)	[]	[]	[]
Commitments (#)	[]	[]	[]
Opened labels total (#)	[]	[]	[]
Calldata size (bytes)	[]	[]	[]
Gas used	[]	[]	[]

5.1 Performance Evaluation

Ethereum performance is shown in Table 4. This is based on a benchmark election instance with $B = 100,000$ ballots and $C = 8$ candidates. This yields $P = BC = 800{,}000$ ballot-candidate positions, which we pad to the next power-of-two $n = \mathsf{pow2}(P) = 2^{20} = 1{,}048{,}576$ to define the evaluation domain $H = \{\omega^0, \ldots, \omega^{n-1}\}$. For this election, we allocate 10% of ballots to print audits.

6 Concluding Remarks

Zeeperio is a succinct proof of an Eperio-encoded election that can be autonomously verified by a smart contract. Even if the reader does not buy the notion of Realcryptik, that not hiding raw ballots from the EA can be a stepping stone to full E2E-V elections, Zeeperio is still useful as a drop-in (or complimentary) mechanism for standard Eperio. Eperio (like Scantegrity and Punchscan) makes a 'blackbox assumption' for ballot privacy that can be extended to Zeeperio. This leads to a natural path for vendors to follow: get comfortable with Zeeprio and then run it in a TEE for full E2E-V.

It was difficult to adequately explain Zeeperio in a conference-length paper. We note that a full version of this work, in the form of a thesis, explains each constraint in detail, offers broader discussion of the implementation, and provides sketches of security arguments.[3] The surface area of the security analysis is small. On one side, Eperio has already been shown to satisfy end-to-end verifiability in both a cryptographic treatment [**?**] and a formal-methods treatment [**?**]. On the other side, modern PIOP-based zk-SNARK constructions justify proving collections of algebraic constraints by reducing them to vanishing-polynomial checks. The remaining gap is therefore narrow but essential: do our specific vanishing constraints characterize the same relation that Eperio intends to enforce?

References

1. Adida, B.: Helios: web-based open-audit voting. In: USENIX Security Symposium. pp. 335–348 (2008)

[3] Link withheld for anonymity.

2. Adida, B., Marneffe, O.d., Pereira, O., Quisquater, J.J.: Electing a university president using open-audit voting: Analysis of real-world use of Helios. In: EVT/WOTE (2009)
3. Basin, D., Dreier, J., Giampietro, S., Radomirović, S.: Verifying table-based elections. In: ACM CCS (2021)
4. Bell, S., Benaloh, J., Byrne, M.D., DeBeauvoir, D., Eakin, B., Kortum, P., McBurnett, N., Pereira, O., Stark, P.B., Wallach, D.S., Winn, M.: {STAR-Vote}: A secure, transparent, auditable, and reliable voting system. In: EVT/WOTE (2013)
5. Boneh, D., Fisch, B., Gabizon, A., Williamson, Z.: A simple range proof from polynomial commitments (2019), https://hackmd.io/@dabo/B1U4kx8XI
6. Brassard, G., Chaum, D., Crépeau, C.: Minimum disclosure proofs of knowledge. J. Comput. Syst. Sci. **37**(2), 156–189 (1988)
7. Bünz, B., Fisch, B., Szepieniec, A.: Transparent snarks from dark compilers. In: EUROCRYPT (2020)
8. Carback, R.T., Chaum, D., Clark, J., Conway, J., Essex, A., Hernson, P.S., Mayberry, T., Popoveniuc, S., Rivest, R.L., Shen, E., Sherman, A.T., Vora, P.L.: Scantegrity II municipal election at Takoma Park: the first E2E binding governmental election with ballot privacy. In: USENIX Security Symposium (2010)
9. Chaum, D., Carback, R., Clark, J., Essex, A., Popoveniuc, S., Rivest, R.L., Ryan, P.Y.A., Shen, E., Sherman, A.T.: Scantegrity II: end-to-end verifiability for optical scan election systems using invisible ink confirmation codes. In: EVT (2008)
10. Chiesa, A., Hu, Y., Maller, M., Mishra, P., Vesely, N., Ward, N.: Marlin: Preprocessing zksnarks with universal and updatable srs. In: EUROCRYPT (2020)
11. Culnane, C., Ryan, P.Y., Schneider, S., Teague, V.: vvote: a verifiable voting system. ACM Transactions on Information and System Security (TISSEC) **18**(1), 1–30 (2015)
12. Essex, A., Clark, J., Hengartner, U., Adams, C.: Eperio: Mitigating technical complexity in cryptographic election verification. In: EVT/WOTE (2010)
13. Gabizon, A., Williamson, Z.J., Ciobotaru, O.: Plonk: Permutations over lagrange-bases for oecumenical noninteractive arguments of knowledge. Cryptology ePrint Archive (2019)
14. Goldreich, O., Micali, S., Wigderson, A.: How to prove all np statements in zero-knowledge and a methodology of cryptographic protocol design. In: CRYPTO (1986)
15. Hao, F., Ryan, P.Y.: Real-world electronic voting: Design, analysis and deployment. CRC Press (2016)
16. Kate, A., Zaverucha, G.M., Goldberg, I.: Constant-size commitments to polynomials and their applications. In: ASIACRYPT. pp. 177–194. Springer (2010)
17. Kusters, R., Truderung, T., Vogt, A.: Clash attacks on the verifiability of e-voting systems. In: IEEE Symposium on Security and Privacy (2012). https://doi.org/10.1109/SP.2012.32
18. Maller, M., Bowe, S., Kohlweiss, M., Meiklejohn, S.: Sonic: Zero-knowledge SNARKs from linear-size universal and updatable structured reference strings. In: CCS (2019)

A Orphaned Text

Zeeperio shifts from statistical audits to cryptographic arguments of knowledge. At a high level, Zeeperio uses the Poly-IOP model to encodes the entire state of an election as a small set of polynomials. Instead of writing a generic arithmetic circuit, Zeeperio works directly at the polynomial level to design custom constraints for ballot structure, sums over ballots, and tallies. Then, using the KZG commitment scheme, Zeeperio allows the Election Authority (EA) to generate a single, succinct proof that the election outcome is correct. This proof is constant in size and verification time, regardless of how many voters cast their ballots.

This efficiency allows for the verification logic to be encoded into an Ethereum smart contract. Instead of requiring observers to download data and run software, the *public audit* becomes a transaction on a the blockchain. If the smart contract accepts the proof, the election tally is mathematically guaranteed to be consistent with the committed ballots. This lowers the barrier to verification from running complex cryptographic software to checking a simple status flag on the Ethereum block explorer.

A.1 Threat Model and Contributions

Before detailing the protocol mechanics, we formalize the operational environment and the specific contributions of this work relative to the state of the art.

A.2 The RealCryptik Threat Model

Zeeprio adopts the *RealCryptik* threat model. Traditional E2E literature often models the EA as a powerful adversary that must be prevented from learning individual votes at all costs. However, this model often ignores the reality of elections. In many scenarios, the EA has the full capability to deanonymize votes.

The RealCryptik model explicitly assumes that the EA **has access to who voted for whom**. Under this assumption, the primary security goal shifts from absolute privacy to absolute **integrity**. The system must guarantee that even an EA with full view of voter choices cannot manipulate an election without detection. This model reflects the reality of *black box* vendors while enforcing the verification constraints necessary for public trust. While public privacy is maintained through zero-knowledge proofs, the protocol also prevents the EA from manipulating the outcome based on their knowledge.

A.3 Contributions

This work presents the following contributions.

Succinct universal verification: All election constraints are encoded as a single Poly-IOP relation and proven via a zkSNARK proof, independent of the number of voters.

Enhanced privacy and dispute resolution: Confirmation codes are committed to a single polynomial and are only selectively opened when needed. After a voter has cast their ballot, they receives a receipt containing a confirmation code to which they can check it appears in the public bulletin board. This is done in a way that prevents the receipt from being used as a proof of how the voter voted.

On-chain verification: The final proof is verified by an Ethereum smart contract. This removes the need for a proprietary verifier software and anchors the election result in a public, tamper-evident ledger.

B Preliminaries

We now introduce the cryptographic tools used in Zeeperio.

B.1 Polynomial interactive oracle proofs

A polynomial interactive oracle proof (Poly-IOP) is an interactive proof where the prover sends commitments to polynomials and the verifier has oracle access to these polynomials through evaluation queries. Formally, a Poly-IOP protocol for a relation $\mathcal{R}$ between public input x and witness w follows:

(i) The prover, given x and w, computes polynomials $f_1, \ldots, f_m$ and sends commitments to them.
(ii) The verifier sends random challenges $r_1, \ldots, r_\ell$.
(iii) The prover answers with evaluations $f(r_\ell)$ and proofs of correct evaluation.
(iv) The verifier checks the evaluations satisfy a system of polynomial identities and the evaluation proofs are valid.

If all checks pass, the verifier accepts the statement *there exists w such that* $(x, w) \in \mathcal{R}$.

B.2 KZG polynomial commitments

Zeeperio uses the KZG (Kate-Zaverucha-Goldberg) polynomial commitment scheme. We recall its interface at a high level. A trusted setup is used to produces a structured reference string (SRS) consisting of group elements

$$\{g, g^\tau, g^{\tau^2}, \ldots, g^{\tau^d}\} \in G_1, \qquad \{h, h^\tau\} \in G_2,$$

for a secret τ.

Given a polynomial $f(X) = \sum_{i=0}^{d} f_i X^i$, the commitment is

$$\mathsf{Com}(f) = \sum_{i=0}^{d} f_i[\tau^i] \in G_1.$$

To open $\mathsf{Com}(f)$ at a point $z \in \mathbb{F}$, the prover computes $y = f(z)$ and the quotient

$$q(X) = \frac{f(X) - y}{X - z}$$

and returns an opening proof $\pi = \mathsf{Com}(q)$. The verifier checks that

$$e(\mathsf{Com}(f) - y, h) = e(\pi, [h^\tau - z]),$$

where $e : G_1 \times G_2 \to G_T$ is a bilinear pairing.

In Zeeperio we use KZG both to commit to the election polynomials. We combine multiple constraints into a single quotient polynomial using random coefficients and prove that it vanishes on Ω by checking a single evaluation.

B.3 Zero-knowledge and Fiat–Shamir

We can apply the Fiat-Shamir heuristic to the Poly-IOP to make the protcol non-interactive. This can be done by replacing the verifier's random challenges by outputs of a hash function on the transcript. To achieve zero-knowledge, we add random blinding polynomials to all witness polynomials before committing, and we ensure that any information opened to the verifier is public. Therefore, the only meaningful information revealed by the proof is that the constraints are satisfied, while keeping the underlying ballot structure and confirmation codes hidden.

B.4 Ethereum verification model

Finally, we briefly outline the on-chain verification model. Ethereum provides precompiled contracts for the BN254 pairing and scalar multiplication. The verification contract uses the SRS and the structure of the combined pairing equation corresponding to the KZG openings. Given the public input and the proof, the contract performs a simple pairing check. If the check passes, the election result has been cryptographically verified.

C The Zeeperio Protocol

Election data and domain. We consider an election with $\mathcal{B}$ ballots and $\mathcal{C}$ candidates, with $\mathcal{P} = \mathcal{B} \cdot \mathcal{C}$ ballot-candidate positions listed in canonical order. Let $i = j \cdot \mathcal{C} + k$ for ballot index $j \in \{0, \ldots, \mathcal{B} - 1\}$ and candidate index $k \in \{0, \ldots, \mathcal{C} - 1\}$. The EA forms the following columns, each of length $\mathcal{P}$.

1. **Ballot Column (B_{ID}):** Identifies the unique ballot $\mathcal{B}_j$.
2. **Confirmation Code Column ($\mathsf{C}_{\mathbf{confirm}}$):** A unique confirmation code issued to each position. The voter receives the code corresponding to their selection.
3. **Mark Column ($\mathsf{M}_{\mathsf{ark}}$):** Indicates if the position was selected for vote.

4. **Audit Column ($\mathsf{A_{udit}}$):** Indicates if the ballot $\mathcal{B}_j$ was selected for audit.

For efficient FFT operations, we require an evaluation domain size n that is a power of two. We set $n = 2^t$ where $t = \lceil \log_2 \mathcal{P} \rceil$. The EA pads the columns with zeros for indices i where $\mathcal{P} \leq i < n$. The columns $\mathsf{M_{ark}}$ and $\mathsf{C_{onfirm}}$ are witness data and are not published.

Let $\mathbb{F}$ be the prime field used by the commitment scheme and let $\Omega = \{\omega^0, \ldots, \omega^{n-1}\}$ be an evaluation domain with primitive nth root ω. Each column is interpolated into a polynomial $F(X) \in \mathbb{F}[X]_{<n}$ satisfying $F(\omega^i) = F[i]$ on Ω.

Commitments and Fiat-Shamir. Zeeperio bases itself as a Poly-IOP using the KZG polynomial commitments with $\mathsf{KZG.Commit}(F)$, $\mathsf{KZG.Open}(F, z)$, and $\mathsf{KZG.Verify}(C, z, y, \pi)$, and derives all verifier challenges via Fiat-Shamir.

Zero-knowledge. We use the hiding variant of KZG commitments (commit as $g^{F(\tau)}h^r$), and off-domain masking (randomize $F'(X) = F(X) + \rho \cdot Z_H(X)$ where $Z_H(X) = X^n - 1$ vanishes on Ω), so commitments and openings do not leak vote or receipt information.

C.1 Setup

In the setup phase, the EA performs the following steps.

(S1) Obtains KZG SRS from a publicly run Ethereum Powers-of-Tau ceremony $\mathsf{SRS} = (g, g^\tau, g^{\tau^2}, \ldots, g^{\tau^d})$, supporting degree $\leq d$ commitments.
(S2) Fixes the finite field $\mathbb{F}_q$, root-of-unity domain Ω of size n, and encoding of election data into polynomials.
(S3) Generates a set of unique confirmation codes and assigns each $\mathsf{C}_c onfirm$ code to each position on the ballot.
(S4) Deploys an Ethereum smart contract that uses the SRS and the verification logic for the Zeeperio proof.

The protocol uses a universal SRS from a Powers-of-Tau ceremony,

C.2 Main Election Proof

Goal. The EA proves in zero-knowledge that the committed election columns satisfy:

- well-formed audit marks
- well-formed cast marks
- audit/mark exclusion
- published tally is consistent with the mark column

Constraint family as vanishing checks. We define a family of low-degree polynomials $\{P_i(X)\}$ such that the election is valid iff

$$P_i(\omega^t) = 0 \quad \forall \omega^t \in \Omega \text{ and for all } i.$$

These constraints encode the following, each as a polynomial identity that vanishes on Ω:
Audit column $\mathsf{A_{udit}}$:

1. The padded values must be set to $\mathsf{A_{udit}}[i] = 0$ for $i \geq P$.
2. All values should either be set to 1, or 0 $\mathsf{A_{udit}}[i] \in \{0, 1\}$.
3. For each ballot j, the $\mathcal{C}$ entries $\mathsf{A_{udit}}[j\mathcal{C}], \ldots, \mathsf{A_{udit}}[j\mathcal{C} + \mathcal{C} - 1]$ are equal. Additionally, the block-sum for each $\mathsf{A_{udit}}$ block is $\in \{0, C\}$.
4. The number of 1s in $\mathsf{A_{udit}}$ is equal to the published number of audited ballots $\mathsf{Sum_A} \cdot \mathcal{C}$ using an accumulator constraint.

Mark column $\mathsf{M_{ark}}$:

1. The padded values must be set to $\mathsf{M_{ark}}[i] = 0$ for $i \geq P$.
2. All values should either be set to 1, or 0 $\mathsf{M_{ark}}[i] \in \{0, 1\}$.
3. The block-sum for each $\mathsf{M_{ark}}$ block is $\in \{0, C\}$.
4. The total number of marks is equal to the published number of ballots cast $\mathsf{Sum_M}$, enforcing one mark per cast ballot using an accumulator.

Audit and Mark Exclusion: No ballot must be both audited and cast $\mathsf{A_{udit}}[i] \cdot \mathsf{M_{ark}}[i] = 0$ for all $i < n$.
Tally correctness: For each candidate $c \in \{0, \ldots, \mathcal{C} - 1\}$, the published tally Sum_c is equal to the sum of $\mathsf{M_{ark}}[i]$ over the stream $\{c, c+\mathcal{C}, c+2\mathcal{C}, \ldots\}$, enforced by a strided accumulator (shift by $\omega^{\mathcal{C}}$) with boundary conditions for the first/last stream elements.

Batched quotient argument. Let $k = 17 + \mathcal{C}$ be the number of vanishing polynomials. Using Fiat-Shamir, derive $\alpha \xleftarrow{\$} \mathbb{F}$ and define the batched vanishing polynomial

$$P_{\mathsf{vanish}}(X) = \sum_{i=1}^{k} \alpha^{i-1} P_i(X).$$

Then form the quotient

$$Q(X) = \frac{P_{\mathsf{vanish}}(X)}{Z_H(X)} \quad \text{where } Z_H(X) = X^n - 1.$$

If all constraints hold on Ω, $Q(X)$ exists as a polynomial.

Proof. The EA commits to the masked witness and auxiliary polynomials required by the constraints ($\mathsf{A_{udit}}, \mathsf{M_{ark}}$ and the block-sum/accumulator polynomials) and to Q, then opens them at a single Fiat-Shamir point $\zeta \in \mathbb{F}$.

Verification. From the opened values at ζ, the verifier reconstructs $P_{\mathsf{vanish}}(\zeta)$ and checks

$$P_{\mathsf{van}}(\zeta) \stackrel{?}{=} Q(\zeta) \cdot (\zeta^n - 1),$$

accepting iff the check passes under $\mathsf{KZG.Verify}$.

C.3 Voter Verification Proofs

Voter verification is a *separate* zero-knowledge proof from the main election proof, but it reuses the same setup and the same committed election polynomials.

Print audit (spoiled ballots). Given a spoiled ballot $\mathcal{B}_j$, the EA opens the commitments for the corresponding index block $\{j\mathcal{C}, \ldots, j\mathcal{C} + \mathcal{C} - 1\}$ to show the ballot was audited and not cast. There is no need to satisfy the receipt-freeness requirement for audited ballots.

Ballot inclusion (receipt check without revealing the choice). To show a voter's receipt code corresponds to a marked position without revealing which candidate was marked, the EA proves a permutation relation over tuples. For each i, form $(b_i, c_i, m_i) = (\mathsf{B_{ID}}[i], \mathsf{C_{confirm}}[i], \mathsf{M_{ark}}[i])$ and a shuffled copy $(b'_i, c'_i, m'_i) = (b_{\sigma(i)}, c_{\sigma(i)}, m_{\sigma(i)})$ for a secret shuffle σ. Using Fiat-Shamir, sample $\alpha, r \stackrel{\$}{\leftarrow} \mathbb{F}$, encode tuples into scalars $v_i = b_i + \alpha c_i + \alpha^2 m_i$ and $v'_i = b'_i + \alpha c'_i + \alpha^2 m'_i$, define $V_1[i] = r - v_i$ and $V_2[i] = r - v'_i$, and enforce multiset equality via a grand product accumulator argument. All resulting checks are expressed as a small set of vanishing polynomials, batched with a random β, and verified with a single quotient test exactly as in the main proof.

After the permutation proof, the EA reveals the *mathcalC* tuples in the shuffled list whose ballot ID equals $\mathcal{B}_j$ via $\mathsf{KZG.Open}$. Exactly one tuple has $m' = 1$, establishing inclusion while the shuffle hides which candidate position was marked.

Receipt correctness (prove a disputed code is not *on ballot $\mathcal{B}_j$).* Let the disputed code be C. The EA defines a helper indicator polynomial $D(X)$ whose evaluations satisfy $D(\omega^i) = 1$ iff $\mathsf{C_{confirm}}[i] = C$ using *Fermat's Little Theorem.* Let $\mathsf{Sel}_j(X)$ be the public selector that is 1 on indices $\{j\mathcal{C}, \ldots, j\mathcal{C} + \mathcal{C} - 1\}$ and 0 elsewhere. The EA proves

$$z \;=\; \sum_{i=0}^{n-1} \mathsf{Sel}_j(\omega^i) \cdot D(\omega^i) \;=\; 0.$$

This implies no confirmation code on ballot $\mathcal{B}_j$ is equal to the disputed code C without revealing any of the ballot's codes.

D Security Analysis

D.1 Completeness

Theorem 1 (Completeness). *If an honest Prover $\mathcal{P}$ computes the accumulators $Acc_{T_1}(X)$ and $Acc_{T_2}(X)$ according to the backward accumulation rule defined in the protocol, then the Verifier $\mathcal{V}$ accepts with probability 1.*

Proof. The honest prover constructs the backward accumulator such that $Acc_{T_1}(\omega^i) = \prod_{j=i}^{n-1} T_1(\omega^j)$. The protocol enforces constraints via a batched vanishing polynomial $P_{vanish}(X)$.

1. **Boundary Consistency:** At the last element $X = \omega^{n-1}$, the accumulator is defined as $Acc_{T_1}(\omega^{n-1}) = T_1(\omega^{n-1})$. Consequently, the term $(Acc_{T_1}(X) - T_1(X))$ is zero at this point, satisfying the boundary constraint $P_{vanish1}$.
2. **Recursive Consistency:** For all $X \in \Omega$ except $X = \omega^{n-1}$, the recursive definition holds: $Acc_{T_1}(\omega^i) = T_1(\omega^i) \cdot Acc_{T_1}(\omega^{i+1})$. This ensures the constraint $P_{vanish3}$ evaluates to zero on the relevant domain.
3. **Product Equality:** Since the tuples are permutations, the total products over Ω are equal. Thus, $Acc_{T_1}(1) = \prod_{x\in\Omega} T_1(x) = \prod_{x\in\Omega} T_2(x) = Acc_{T_2}(1)$, satisfying the equality constraint $P_{vanish5}$.

Since all individual constraint polynomials vanish on Ω, $P_{vanish}(X)$ is divisible by the vanishing polynomial $Z_H(X) = X^n - 1$. Therefore, the quotient check $P_{vanish}(\zeta) - Q(\zeta)Z_H(\zeta) \stackrel{?}{=} 0$ holds for any $\zeta \in \mathbb{F}_q$. □

D.2 Knowledge Soundness

Theorem 2 (Knowledge Soundness). *The protocol satisfies knowledge soundness in the Algebraic Group Model. If a malicious prover $\mathcal{P}$ convinces $\mathcal{V}$ with non-negligible probability ϵ, there exists an efficient Extractor $\mathcal{E}$ that outputs polynomials $T(X)$ and $T'(X)$ such that the set of their evaluations on Ω are permutations of each other.*

Proof. In the AGM, the Extractor $\mathcal{E}$ observes the commitments sent by $\mathcal{P}$. Since $\mathcal{P}$ outputs valid commitments $[Acc_{T_1}], [Acc_{T_2}]$, it must know the underlying polynomials.

If $\mathcal{V}$ accepts, the equation $P_{vanish}(\zeta) = 0$ holds at a random challenge $\zeta \leftarrow \mathbb{F}_q$. By the Schwartz-Zippel Lemma, this implies that $P_{vanish}(X)$ is divisible by $Z_H(X)$ with overwhelming probability. We unroll these constraints to prove the validity of the statement:

1. **Unrolling the Accumulator:** From the boundary constraint ($P_{vanish1} \equiv 0$), we have $Acc_{T_1}(\omega^{n-1}) = T_1(\omega^{n-1})$. From the constraint ($P_{vanish3} \equiv 0$), for any $x \in \Omega$ except $X = \omega^{n-1}$, $Acc_{T_1}(x) = T_1(x) \cdot Acc_{T_1}(\omega x)$. With this

backwards accumulation, we determine that $Acc_{T_1}(1)$ stores the complete product:

$$Acc_{T_1}(1) = \prod_{i=0}^{n-1} T_1(\omega^i) \tag{1}$$

Similarly, we derive $Acc_{T_2}(1) = \prod_{i=0}^{n-1} T_2(\omega^i)$.

2. **Equality of Multisets:** The final constraint ($P_{vanish5} \equiv 0$) ensures $Acc_{T_1}(1) = Acc_{T_2}(1)$. Substituting the definitions of T_1 and T_2:

$$\prod_{x \in \Omega} (r - T(x)) = \prod_{x \in \Omega} (r - T'(x)) \tag{2}$$

This equation implies equality between two polynomials in the variable r. Since r is chosen uniformly at random by the verifier *after* the commitments to $T(X)$ and $T'(X)$, the equality holds identically with overwhelming probability. Therefore, these polynomials are considered to be *set equal*. Thus, T' is a valid permutation of T.

□

D.3 Honest-Verifier Zero-Knowledge

Theorem 3 (HVZK). *The protocol is Honest-Verifier Zero-Knowledge. There exists a Simulator $\mathcal{S}$ that outputs a transcript indistinguishable from a real prover's trace.*

Proof. The Simulator $\mathcal{S}$ operates as follows:

1. **Challenge Sampling:** $\mathcal{S}$ samples random challenges $r, \beta, \zeta \leftarrow \mathbb{F}_q$ exactly as the honest verifier would.
2. **Simulate Commitments:** $\mathcal{S}$ outputs random group elements for $[Acc_{T_1}], [Acc_{T_2}], [T_1], [T_2]$. Due to the hiding property of KZG (with blinding), these are indistinguishable from real commitments.
3. **Simulate Evaluations:**
   - $\mathcal{S}$ chooses random values for evaluations $\bar{T}_1, \bar{T}_2, \overline{Acc}_{T_1}, \overline{Acc}_{T_2}$ at ζ and required shifted points (e.g., $\omega\zeta$). This is valid because the blinded polynomials act as One-Time Pads at any point $\zeta \notin \Omega$.
   - $\mathcal{S}$ must ensure the verification equation holds: $P_{vanish}(\zeta) - Q(\zeta)Z_H(\zeta) = 0$.
   - $\mathcal{S}$ computes the value of $P_{vanish}(\zeta)$ using the randomly sampled evaluations and the formulas for Z_1, Z_2, Z_3 evaluated at ζ.
   - $\mathcal{S}$ then *sets* the evaluation of the quotient polynomial $\bar{Q} = Q(\zeta)$ specifically to satisfy the equation:

$$\bar{Q} = \frac{P_{vanish}(\zeta)}{Z_H(\zeta)}$$

   Since ζ is random, $Z_H(\zeta) \neq 0$, making this division valid.
4. **Transcript Consistency:** The tuple of commitments, challenges, and evaluations $(\ldots, \bar{Q})$ satisfies the algebraic verification check by construction. Because the evaluations of blinded polynomials are uniformly distributed, the simulated transcript is statistically indistinguishable from a real transcript.

End-to-end Verifiable Zeeperio pipeline.

Election table → witness columns → evaluations on H → KZG commitments → openings at $\{\zeta, \zeta\omega, \ldots\}$ → on-chain verification (pairings + constraints).

Fig. 2. E2E Verifiable proof pipeline.

E Implementation and Performance Evaluation

We describe a production-ready implementation of Zeeperio that takes the polynomial relations from (Section 3- section 4) and verifies them on Ethereum under gas constraints. The system has two parts:

- an off-chain prover implemented in Rust that constructs the necessary witness columns, forms KZG commitments, and produces batched openings.
- an on-chain verifier implemented as a Solidity smart contract that recomputes Fiat–Shamir challenges, checks KZG openings using BN254 pairings, and validates the aggregated vanishing identities at a single challenge point ζ.

E.1 Implementation

Off-chain prover. An election instance is defined by $\mathcal{B}$ ballots and $\mathcal{C}$ candidates, yielding $\mathcal{P} = \mathcal{BC}$ ballot-candidate positions. We pad to the next power of two $n \geq P$ and define the evaluation domain $H = \{\omega^0, \ldots, \omega^{n-1}\}$ where ω is an nth root of unity. Each length-n column is represented as evaluations on H, and the prover deterministically derives the auxiliary columns required by the protocol.

All constraints are enforced via vanishing identities over H. For each constraint we form a polynomial $V_i(X)$ intended to vanish on H, and we define the corresponding quotient $Q_i(X)$ such that

$$V_i(X) \;=\; Q_i(X) \cdot Z_H(X),$$

where $Z_H(X) = X^n - 1$ is the vanishing polynomial for H. Instead of proving each constraint separately, the prover aggregates them using a Fiat-Shamir challenge α and proves a single identity at ζ:

$$p_{\mathsf{vanish}}(\zeta) \;:=\; \sum_i \alpha^i V_i(\zeta) \;=\; Q(\zeta) \cdot Z_H(\zeta),$$

where Q is the aggregated quotient polynomial committed by the prover.

To satisfy the zero-knowledge property, the prover applies *hiding KZG* commitments, where it adds a blinding term derived from a $\gamma \cdot g$ SRS basis. Additionally, the prover uses *Z-masking* for witness polynomials not meant to be revealed off-domain, and randomizes them as $f'(X) = f(X) + \rho(X)\, Z_H(X)$, which preserves evaluations on H.

Fiat-Shamir transcript. Challenges are derived non-interactively by hashing public inputs and serialized commitments in a fixed order. The transcript yields (α, ζ) for the main proof and additional challenges for the inclusion/shuffle proof. Both prover and verifier must share *exact* serialization rules (little-endian field encoding and compressed group encoding), otherwise valid proofs will be rejected.

Batched openings and ABI export. To keep calldata and gas predictable, the prover batches multiple openings at a common point using a folding scalar γ and outputs a single witness group element per point. The exported ABI payload contains public inputs, labeled commitments, a list of opening batches (each with point, γ, labels, opened values, and blinding scalars), and the transcript scalars.

On-chain verifier. The verifier exposes three entrypoints, `verifyMain(bytes)`, `verifyInclusion(bytes)`, and `verifyReceipt(bytes)`. Each entrypoint decodes calldata into a structured proof, recomputes transcript challenges from the decoded statement and commitments, and rejects if challenges do not match.

For each opening batch at point x, the contract folds the relevant commitments and values, then checks a single KZG pairing equation via the BN254 pairing precompile. After all the openings have been validated, the contract reconstructs the required evaluations, computes the proof $p_{\mathsf{vanish}}(\zeta)$, and checks

$$p_{\mathsf{vanish}}(\zeta) = Q(\zeta)\,(\zeta^n - 1).$$

To keep the EVM logic minimal, commitments and opened values are tagged by 16-bit labels which help the verifier locate required evaluations via simple scans over decoded arrays.

E.2 Performance Evaluation

We evaluate Zeeperio along the three axes: (i) prover time, (ii) verifier time (including decoding), and (iii) calldata size. We report metrics separately for the main proof and the two voter verification proofs.

Benchmark configuration As shown in Table 3, we report metrics for a single election instance with $B = 100,000$ ballots and $C = 8$ candidates. This yields $P = BC = 800{,}000$ ballot-candidate positions, which we pad to the next power-of-two $n = \mathsf{pow2}(P) = 2^{20} = 1{,}048{,}576$ to define the evaluation domain $H = \{\omega^0, \ldots, \omega^{n-1}\}$. For this election, we allocate 10% of ballots to print audits.

Metrics reported We report three deployability metrics for each proof kind: (i) prover time, (ii) proof/calldata size, and (iii) on-chain gas.

Proof size. We report: number of commitments, number of opening points, total opened labels, and ABI calldata length in bytes.

On-chain cost. We report gas used by `verifyMain`, `verifyInclusion`, and `verifyReceipt`.

Table 3. Fixed election instance used for benchmarking.

Parameter	**Value**
Ballots ($\mathcal{B}$)	$100,000$
Candidates ($\mathcal{C}$)	8
Positions ($\mathcal{P} = \mathcal{BC}$)	$800,000$
Padding (n)	$2^{20} = 1,048,576$
Print-audit allocation	10%
Audited ballots ($\mathcal{B}_{\mathsf{audit}}$)	$10,000$

Table 4. Implementation metrics for $B = 100{,}000$, $C = 8$

Metric	**Main**	**Inclusion**	**Receipt**
Prover time (s)	[]	[]	[]
Commitments (#)	[]	[]	[]
Opened labels total (#)	[]	[]	[]
Calldata size (bytes)	[]	[]	[]
Gas used	[]	[]	[]